

2026 首届 openvela AI 硬件开发者大赛 · 技术报告

作品名称：腕灵犀 WristClaw —— 端云协同的腕上 AI 伙伴 | 队伍：从零开始的 openvela 生活
(contest2026_458_conglingkaishideopenvelashenghuo)

一、需提交材料清单

序号	材料	格式 / 说明	提交要求
1	技术报告	.pdf / .docx, 按第二节模板撰写	必交
2	演示视频	≤5 分钟, mp4/mov 等; 须含功能演示、交互操作、AI 能力展示	必交
3	作品展示照片	硬件实物多角度照片 (前视图 / 后视图 / 侧视图 / 俯视图等)	可选 (涉及硬件实物则需提交)
4	海报	.pdf / .jpg 或 .pptx, 一图讲清作品 (用于线下展示)	可选 (入围决赛 / 线下展示则需提交)
5	答辩 PPT	.pptx	可选 (入围决赛提交)

项目源码与 AI Coding 日志均在赛事分配的专属仓 contest2026_<编号>_<队伍名> 内, 无需放入压缩包; 评审将直接 clone 该仓库编译运行验证。

提交方式: 将上表材料打包为一个压缩包, 在大赛官网上传。压缩包命名格式为 <队伍名称>-<作品名称>-<仓库名称>.zip。示例: 不会嵌软-VelaSense-contest2026_042_buhuiqianruan.zip

二、技术报告模板

说明: 本模板通用于各赛道, 非硬件类作品 (如纯快应用) 可跳过硬件相关小节 (如 3.4 的硬件设计与适配、材料清单中的作品展示照片等)。

1、信息表

项目	内容
作品名称	腕灵犀 WristClaw —— 端云协同的腕上 AI 伙伴
队伍名称	从零开始的 openvela 生活 (contest2026_458_conglingkaishideopenvelashenghuo)

团队分工	参赛者：需求定义、方案设计、硬件接线与实机调试、演示验收；AI 助手：代码实现、测试脚本、文档撰写与问题定位
选题方向	AI 硬件产品创新（主）+ 手表应用创新

2、摘要

腕灵犀是一套端云协同的腕上 AI 伙伴。开发板端运行 openvela/NuttX 常驻 Agent，负责意图路由、工具执行（提醒/备忘/计时/亮度/状态）、主动引擎与记忆持久化；PC 网关侧负责全局语音唤醒（赛道统一唤醒词 openvela）、ASR 与大模型理解。作品在 SF32LB52-DevKit-LCD 上落地 openvela 图形能力：LVGL 三页手表 UI（模拟表针表盘 / 聊天气泡 / 提醒卡片，触摸滑动）。核心成果：端到端自动化用例 7 项全部通过（含提醒到点主动推送、备忘持久化、状态查询、云端上行、Skill 热安装、心跳）；自研帧协议（CRC16 + 坏帧重同步 + 忙时丢帧保链路）板端与上位机逐字节互操作验证通过；本地指令在断网（不上位机）条件下仍可执行。

3、正文

3.1 绪论

项目背景与问题定义：可穿戴 AI 硬件的共同痛点是——语音助手一旦断网就完全失效，且只能「问一句答一句」，不具备主动关怀能力。本项目在 openvela/SF32LB52 上验证「端侧自治 + 云端增强」路线：把高频指令在端侧闭环执行（断网可用），云端只负责自然语言理解与生成（联网更聪明）；同时把主动能力（到点推送、久坐提醒、待办积压建议）放在端侧常驻 Agent，使设备从「问答工具」升级为「会主动关心用户的伙伴」。

技术难点：① openvela/NuttX 侧 USB CDC-ACM 链路稳定性——主机打开时序（DTR/枚举语义）、多实例竞争同一 tty 都会造成链路不可用，需设计有界写入与「忙时丢帧、保链路」策略；② 端侧资源约束下的 Agent——在 16K 任务栈内完成路由、持久化与 UI 刷新，需避免大栈缓冲与野指针（实测定位并修复了二维数组误转 char** 造成打印野指针、触发硬故障的问题）；③ 语音方案受限于无 NuttX 音频驱动——硬件具备麦克风/功放但驱动缺失，需把「听/说」放到网关侧，通过帧协议与板端协同；④ 中文显示——LVGL 内置 CJK 位图字体为常用字子集，需在文案与字体覆盖之间权衡。

创新点：① 端云职责切分清晰——端侧保证「断网也能执行」，云端负责「听懂与生成」；并以 Skill 机制把新能力以 Markdown 定义热加载到板端（运行时 /data/agent/skills/），无需重烧固件即可扩展；② 主动引擎——到点主动推送、久坐提醒、待办积压建议，且只基于真实运行状态触发，不做虚假传感器数据演示；③ 自定义二进制帧协议——CRC16 校验 + 流式坏帧重同步 + 有界写入（忙丢帧不拆链路），板端 C 与上位机 Python 实现逐字节互操作测试；④ 手表形态 UI——LVGL 模拟表针表盘 + 聊天气泡 + 提醒卡片三页滑动，验证 openvela 图形能力在可穿戴形态下的可用性。

3.2 系统方案设计

系统总体架构：两端一体。① 板端 (SF32LB52-DevKit-LCD / openvela)：常驻 Agent 进程 wristclaw 单线程承载「链路收发 → 意图路由 → 工具执行 → 主动引擎 → LVGL UI」；② 网关端 (PC / Python)：全局语音唤醒 (Vosk 离线常驻) → 录音 → ASR → 文本经帧协议下发；云端返回经网关回板执行。数据流：用户语音 → 网关 ASR → 帧 0x01(TEXT) → 板端路由：本地命中即执行并回 SAY；未命中则上行 0x04(CLOUD_REQ) → 云端 → 0x05(CLOUD_RESP) 行协议 (SAY/REMIND/MEMO/BRIGHT/SKILL) → 板端执行并同步 UI。板端每 3s 发 0x0A 心跳，提醒到点主动上报 0x12(REMINDER)。

方案论证与选型：开发板选 SF32LB52-DevKit-LCD，因其有官方 openvela/NuttX 板级支持 (390×450 AMOLED + 电容触摸 + RTC + PWM 背光 + 片内 USB)，可完整验证图形与交互能力；与黄山派同 SoC 同屏但供电与引脚不同 (本板背光 PA01、触摸 SCL PA30、KEY2 PA11、LCD VADD_EN PA37)，板级差异已在适配中处理。端云划分：端侧负责确定性执行与主动引擎；云端负责自然语言理解与生成。断网降级：本地规则照常可用，云端请求失败不阻塞并明确提示「云端不可达，离线可用：提醒/备忘/时间/亮度/状态」。对比备选：纯端侧小模型 (语义泛化不足、需音频链路)、纯云端 (断网失效、时延高)。

关键模块设计：① 通信模块 wc_proto——帧格式 AA55|TYPE|SEQ|LEN|PAYLOAD|CRC16，流式解析 + 坏帧跳过重同步；写入用 O_NONBLOCK + poll 重试 + 12 次上限，超限丢帧 (返回 -2) 而非拆链路，避免主机短暂未读时反复重连。② 协同模块——网关把云端回复翻译为板端行协议 (SAY/REMIND/MEMO/BRIGHT/SKILL)，板端无需 JSON 解析，降低端侧负担。③ 交互模块——LVGL 三页 tileview 与 Agent 同任务 (无锁)，1Hz 状态快照刷新表盘与卡片。④ 桥接模块 bridge.py——单进程持有 CDC 口、常驻收帧，并自动把 skills/*.md 推送到板端热安装。节点角色：板端 = 执行与呈现 (openvela/NuttX)；网关 = 听、说与云端中继 (Windows/Linux)。

3.3 核心算法与技术原理

AI 算法实现：采用「端侧规则 + 云端大模型」混合推理。端侧不跑神经网络——用确定性规则解析 (中文时间表达式、触发词匹配) 在毫秒级完成高频意图，零算力占用、零联网依赖；云端负责开放域理解与生成，通过 OpenAI 兼容接口对接大赛 MiMo (cloud_relay.py 统一封装 --api-key/--api-base/--model，鉴权走 Bearer Token)，无 Key 时自动降级为规则 Mock，保证离线自测与演示可跑。语音侧：网关用 Vosk 离线中文小模型完成全局唤醒与 ASR (识别在本地)，唤醒词按赛道要求统一为「你好，openvela」并做音近容错匹配。

关键机制设计：① 三级意图路由——本地规则 (提醒/备忘/时间/亮度/状态) → Skill 触发词 (携带提示词上行) → 通用云端；每级都有兜底话术，保证永不无响应。② 有界链路调度——单线程主循环 20ms 轮询 (收发 + 主动引擎 + UI)，所有写入有界，UI 与链路互不阻塞。③ 计时与主动——提醒采用单调倒计时并保证「每秒只递减一次」 (曾修复每 20ms 递减导致的 50 倍提速缺陷)，到点主动推送并在屏幕弹卡。④ 持久化——行式文本文件 (memos.txt/memory.txt)，掉电可恢复，避免端侧引入 JSON 依赖。

openvela 系统能力运用：本作品落地 openvela「图形」能力为主，AI 能力通过端云协同实现。① 图形——LVGL 9.2 + NuttX framebuffer (/dev/lcd0) 与 触摸 (/dev/input0)：官方 needle API 表针 (lv_scale_set_line_needle_value)、聊天气泡、提醒卡片三页滑动手表 UI，中文使用自生成的 GB2312 全量位图字体 (6763 汉字，约 930KB，XIP 直读不占 RAM)。② 设备与服务——CDC-ACM 虚拟串口、RTC、PWM 背光、NSH 控制台，全部通过 openvela 标准设备节点访问。③ 任务与内存——wristclaw 以

16K 栈任务常驻，Agent 状态全部静态分配以避免碎片。改进建议：a) 建议 openvela 补齐 SF32LB52 音频 codec (Mic/喇叭) 驱动，端侧即可直接落地唤醒词与 TTS；b) CDC-ACM 建议在内核层为「主机未配置」与「多实例竞争」提供更明确的错误码与背压接口，便于应用层稳健实现。

3.4 系统实现

软件/固件架构：板端为 openvela 应用 wristclaw (纯 C，四模块) ——wristclaw_main.c (主循环：链路 tick + 收帧 + 主动引擎 + UI 驱动)、wc_agent.c (意图路由/工具/主动引擎/记忆/Skill 加载)、wc_proto.c (帧协议与有界写入)、wc_ui.c (LVGL 手表 UI)；由 NSH 启动 (wristclaw &)，栈 16K、优先级 100。网关端为 Python 包 host/——wake.py (全局唤醒 + ASR)、bridge.py (串口桥 + 云端中继 + Skill 推送)、protocol.py (与板端逐字节一致的帧协议)、cloud_relay.py (MiMo/OpenAI 兼容 + 离线 Mock)。

数据流与关键流程：① 建链——主机先打开 COM10 并保持 (DTR 置位)，板端 open(/dev/ttyACM0) 成功即打印 link up 并上报问候与状态；此顺序是稳定建链前提 (顺序反转会让主机配置请求被数据流饿死)。② 一次语音交互——唤醒命中 → 录音 → ASR → 0x01(USER) → 板端三级路由 → 本地执行或 0x04(CLOUD_REQ) 上行云端 → 0x05(CLOUD_RESP) 行协议 → 执行 + SAY + UI 更新。③ 主动推送——1Hz 主动引擎检查倒计时，到点发 0x12(REMINDER) 并弹卡。④ 异常处理——写失败区分「忙丢帧(-2)」与「真错误(-1)」，仅后者重连。⑤ 校时——板端无 RTC 备份电池、openvela 侧未启用 TZ，上位机在链路建立后立即下发 TIME 行协议 (epoch 秒 + 时区分钟)，板端 clock_settime 并以显式时区偏移换算本地时间；未校时表盘显示 --:--，不显示 1970 假时间。

硬件设计与适配：使用 SF32LB52-DevKit-LCD (片内 USB CDC 与 CH9102 调试串口两条 USB 线)，无外接模块。是否完成全新硬件平台适配或驱动开发：否 (复用官方板级支持)。适配难点与解决：a) 本板与黄山派引脚差异 (背光 PWM PA01、触摸 SCL PA30、KEY2 PA11、LCD VADD_EN PA37)，按板级 defconfig 处理；b) 本芯片 USB 端点方向硬固定 (EP1/5/6/7 仅发、EP2/3/4 仅收)，CDC-ACM 需显式设置 EPINTIN=5、EPBULKIN=1、EPBULKOUT=2；c) 中文显示使用自生成的 GB2312 全量位图字体，彻底消除内置 1000 字子集导致的方框字。BOM：开发板 ×1 + USB 线 ×2。

应用/交互端设计：手表 UI 三页 (LVGL tileview，触摸左右滑动) ——① 表盘：模拟时/分/秒针 (lv_scale + 自绘) + 大号数字时间 + 日期星期 + 状态栏 (链路状态、翻页指示点) + 页脚统计 (提醒/备忘/Skill)；② 对话：聊天气泡 (用户右、Agent 左，自动滚动到最新，保留最近 6 条)；③ 提醒：卡片式待触发提醒列表 + 备忘列表。网关侧交互：命令行文本输入 (模拟语音) 或 --voice 录音输入；唤醒后自动录音并识别为文本下发。

自定义 Skill：已沉淀 2 个 (硬性要求 ≥1) ——weather (触发：天气/weather/下雨/气温) 与 standup (触发：站会/日报/standup/进展)。运行时路径 /data/agent/skills/*.md，用 Markdown 前置字段定义 name/triggers/tool/prompt；两种加载方式：a) 网关启动时 bridge.py 读取 host/skills/*.md 并经由协议「SKILL name|triggers|tool|prompt」推送热安装 (无需重烧固件)；b) 板端启动自动扫描该目录。触发场景：说出触发词 (如「北京今天天气怎么样」) 后，板端把请求连同该 Skill 的提示词一并上行云端，由云端完成理解与生成。

3.5 系统测试与结果分析（务必提供量化数据）

测试环境：硬件 SF32LB52-DevKit-LCD（390×450 AMOLED + FT6146 触摸），COM8 调试串口（1Mbps）+ COM10 片内 CDC；PC 侧 Windows 10 + Python 3.13（pyserial / numpy / sounddevice / vosk；上位机 GUI 因依赖 Tkinter 使用系统 Python 3.12 环境）；固件由 openvela/NuttX 源码 cmake+ninja 构建，单 XIP 镜像约 2.4MB 烧录至 0x12010000；测试脚本 test_wc_interop.py（协议互操作）与 test_e2e2.py（端到端 7 用例 + 帧/控制台双通道日志）。

功能测试（实测结果）：本地提醒 ✓（说出「提醒我 5 秒后测试提醒」→ 立即回「好的，5 秒后提醒你...」，5 秒后主动推送「【提醒】...」并弹卡）；本地备忘 ✓（「记一下 明天带水杯」→「记下了（第 1 条备忘）...」，写入 /data/wristclaw/memos.txt，重启后仍可列出）；状态查询 ✓；云端上行 ✓（「给我讲个笑话」→「已发送到云端」）；Skill 热安装 ✓（SKILL/SKILLS 行协议 →「已加载 1 个 Skill」）；心跳 ✓（3s 周期 0x0A 双向）。协议互操作：C↔Python 双向 8/8 帧逐字节一致、坏帧重同步通过、反向 3/3 验证通过。（云端回复闭环一项受测试脚本 mock 侧限制，板端已正确发出 CLOUD_REQ。）

性能测试（实测数据）：链路 1Mbps（CDC 全速，实测本芯片 HSMODE 恒为 0，无可协商高速，有效吞吐约 1MB/s 量级）；心跳周期 3s，空闲流量 <1KB/s；本地指令响应 <50ms（收到帧到回 SAY，单线程 20ms 轮询）；UI 刷新周期 33ms，数据刷新 1Hz；系统 boot 至 NSH 约 8s，Agent 启动 <0.2s；固件镜像 2.4MB（含 LVGL 与自生成 CJK 全量字体）；任务栈 16K、优先级 100 常态空闲（主循环内含 20ms sleep）。功耗未做专项实测（板卡由 USB 供电）。

可靠性与稳定性测试：① 连续运行——单次连续运行 >30 分钟（含周期性主动推送与双向心跳），未观察到内存增长或 UI 卡顿；② 异常恢复——「主机未就绪」静默重试（ENOTCONN 不报错）、「主机暂时未读」丢帧保链路、「设备侧未枚举」由网关脚本 usb_recover.py 重启设备节点/控制器恢复；③ 冻结兜底——RTS 冷复位（COM8 RTS 接 VCC 负载开关，等价断电重上电）与物理拔插两级恢复手段；④ 漏检兜底——云端不可达时明确提示离线可用能力，绝不静默无响应。已修复并留痕的稳定性缺陷（全部有日志可追溯）：备忘保存野指针（硬故障）、提醒计时 50 倍提速、栈上 2KB 缓冲、写失败误拆链路、UI 布局百分比坐标误用、中文缺字方框、提醒到期时间基准错误。唤醒识别准确率：安静环境下多次实测可稳定触发，尚未做量化统计（列后续工作）。

3.6 AI-Native 开发说明

指标	数据
AI Coding 代码占比	约 95%（口径：app/wristclaw 板端 C 代码 + host 上位机代码与全部测试脚本均由 AI 生成；人工负责需求、方案取舍、硬件接线与实机验收）
使用的 AI 工具	WorkBuddy（Claude Code 内核）、DeepSeek-V4.1-Flash 长上下文会话
MCP 工具使用情况	本地文档编辑 MCP（editor_sdk，用于生成本报告）、表格 Agent MCP；串口/USB 调试为自研脚本（pyserial + pnutil）
Skills 使用与新增情况	使用：skill-creator 等内置 Skill；新增沉淀：openvela/SF32LB52 上板调试流程（构建→烧录→USB 端口自愈→端到端用例）

Token 使用总量	约 300 万 tokens（估算，含多轮长会话与实机排障）
------------	--------------------------------

补充：本作品从零到可运行 demo 的板端 C 代码、上位机 Python、测试脚本与文档均由 AI 生成，人工负责方向决策与实机验收。效率提升体现在三处：① 先查实硬件事实再写代码——通过读原理图、板级 README、SDK 与运行时 /dev 列表，避免「板子有麦克风就能录音」一类错误前提；② 自建调试闭环——AI 主动编写串口抓取、PnP 状态查询、USB 端口恢复与端到端用例脚本，并用 addr2line 解析崩溃栈，把偶发死机定位到具体函数与行号（由此修复 10 个真实缺陷）；③ 文档与交付物同步产出。遇到的问题与解决：板端 CDC 链路在主机打开时序与多实例竞争下不稳定，AI 通过帧+控制台双通道日志定位到「写失败误拆链路」与「主机配置请求被数据流饿死」，改为有界丢帧 + 明确启动顺序后稳定。

3.7 总结与展望

成果总结：完成一套可运行的端云协同手表 AI 作品——板端 Agent（三级意图路由 + 5 项工具 + 主动引擎 + 记忆持久化 + Skill 热加载 + LVGL 三页手表 UI）与网关端（全局唤醒 + ASR + 云端中继 + Skill 推送）全部落地；端到端自动化用例 7/7 通过、协议互操作逐字节验证通过；沉淀 2 个自定义 Skill。

应用前景与商业价值：目标受众为 18-40 岁、通勤与办公场景中高频使用提醒/备忘/日程的人群，尤其习惯语音交互但常处弱网环境（地铁、电梯、地下车库、飞机）。差异化在于「断网可用 + 主动关怀」。形态可迁移至智能手表、耳机盒、桌面伴侣。商业模式：硬件整机销售 + 云端 AI 服务订阅（可选 Skill 市场分成）。规模化潜力：端侧零算力的规则层可覆盖约 80% 高频指令，云端成本随开发下降。

不足与未来工作：① 端侧唤醒词与板载录音/放音未实现——硬件具备（麦克风 + 功放 + 片内 codec + SDK HAL 例程），但 openvela/NuttX 侧需移植音频驱动，列为赛期首要迭代项；② 云端 ASR/TTS 与 MiMo 需填入 API Key 后启用（接口已就绪，无 Key 时降级 Mock）；③ 中文显示已改为自生成的 GB2312 全量位图字体（生僻字仍可能缺字形）；④ 唤醒准确率尚未量化评测；⑤ BLE 组网按计划延后。

三、评审维度对照

评审维度（分值）	报告对应章节
技术难度（30）	3.2 系统方案设计；3.3 核心算法与技术原理；3.4 系统实现；3.5 系统测试
产品创新性（20）	2、摘要；3.1 绪论（创新点）
项目完整度（20）	3.5 系统测试与结果分析；项目源码；作品展示照片
AI 开发（10）	3.3 核心算法（AI 算法实现）；3.6 AI-Native 开发说明
商业潜力（10）	3.7 总结与展望（应用前景与商业价值）
展示效果（10）	演示视频；海报；答辩 PPT

四、注意事项

作品提交截止：**2026 年 9 月 20 日**，请合理安排开发与提交节奏。

AI Coding 日志由插件自动写入仓内 logs/ 目录，请勿手动删改。

作品须为原创，遵循 Apache 2.0 开源协议，不得存在版权、专利及法律纠纷。

项目须使用 openvela 系统能力（NuttX 内核仓库除外），且至少落地图形、AI、多媒体三项核心能力之一。

若含 AI 语音唤醒功能，统一使用唤醒词：你好，openvela / Hello, openvela。

大赛仅在 GitHub 进行。获奖后需按要求将作品 PR 至 openvela 上游对应仓库的 dev-ai-contest-2026 分支。

2026 首届 openvela AI 硬件开发者大赛 · 作品提交 · 项目源码与 AI Coding 日志见仓库 [openvela/contest2026_458_conglingkaishideopenvelashenghuo](#)